COLUMBIA UNIVERSITY

Introduction to the Mathematics of Finance (Fall 2025)

Instructor: Prof. Mikhail Smirnov

Calibration and Pricing of Athena Autocalls

Heston Model Using FFT and Monte Carlo

Team Members

Ethan Chemla (EC4003)

Ha Pham (HTP2105)

Jesse Jacob Price (JJP2224)

Ningyuan Tang (NT2504)

Raphael Demare (RD3213)

Vijeet Prasad (VP2580)

December 19th, 2025

1 Introduction

Structured products like the **Athena Autocallable Note** is useful for investors seeking yield enhancement in exchange for specific tail risks. This paper implements a robust pricing framework for a 4-Year Autocallable Note linked to the S&P 500 (SPX).

Valuing such path-dependent instruments with deep out-of-the-money (OTM) barriers requires going beyond the standard Black-Scholes model, which fails to capture the implied volatility skew and leverage effect. To address this, we use the **Heston (1993) stochastic volatility model**. Our methodology proceeds in two phases: first, we calibrate the model parameters to the full market volatility surface using the **Lewis (2001) Fast Fourier Transform (FFT)** method; second, we price the structure using a **Monte Carlo simulation** with the Full Truncation scheme to ensure numerical stability. Finally, we decompose the fair value of the product and analyze its sensitivity to critical risk factors, specifically the spot-volatility correlation.

2 Product Specifications and Market Data

2.1 Athena Autocallable Note Structure

We analyze a **4-Year Athena Autocall Note** on the **S&P 500 (SPX)** issued by BNP Paribas in 2023. The structure features semi-annual observations with a **Snowball coupon** of 4.52% per semester (9.04% p.a.) contingent on the index closing at or above the **100% Autocall barrier**, thus triggering early termination.

To simulate a live pricing environment, we reset the pricing date to the system date while maintaining the original semi-annual schedule and 4-year maturity. We also adjust the coupon to **3.00%** semi-annually. This ensures the note's price is slightly below par to incorporate the issuer's profit margin.

The risk profile is defined by a **60% European Knock-In barrier**. If the note is not autocalled and the index falls below this threshold at maturity ($T = 4$), the capital protection is lifted, exposing the investor to the full equity drawdown (S_T/S_0). This deep out-of-the-money downside risk ($K = 60\%$) requires a model capable of accurately pricing the volatility skew.

2.2 Market Data Construction

Yield Curve (SOFR): We construct the risk-free zero-coupon curve by **bootstrapping SOFR OIS swaps** (Nov 25, 2025 data from Bloomberg). The short end ($T \leq 18m$) uses linear discounting of money market rates, while the long end is solved recursively from par swap rates to generate a continuous term structure (see Appendix A for the curves).

Volatility Surface (SPX): Real-time option chains (save and reuse Dec 3, 2025 data for reproducibility) are processed to ensure a robust Heston calibration. To mitigate liquidity noise and prevent static arbitrage, we:

1. **OTM Splicing:** Construct the surface using exclusively OTM contracts (Puts for $K < S_0$, Calls for $K \geq S_0$).
2. **Implied Dividends:** Extract continuous dividend yields $q(T)$ via Put-Call Parity on liquid ATM pairs.
3. **Quality Checks:** Check butterfly arbitrage and filter quotes to ensure strict monotonicity. Remove illiquid ticks.

3 Mathematical Framework

3.1 Heston Stochastic Volatility Model

To overcome the limitations of the Black-Scholes framework, particularly regarding the volatility smile and leverage effect, we employ the Heston (1993) model. Under the risk-neutral measure, the spot price S_t and variance v_t follow:

$$\begin{aligned} dS_t &= (r-q)S_t dt + \sqrt{v_t} S_t dW_t^S \\ dv_t &= \kappa(\theta - v_t)dt + \sigma\sqrt{v_t} dW_t^v, \quad d\langle W^S, W^v \rangle_t = \rho dt \end{aligned} \tag{1}$$

Parameter Interpretation: κ and θ define the mean-reversion of variance, capturing the clustering of volatility. σ (Vol-of-Vol) controls the kurtosis. The correlation ρ is critical for Autocalls: a negative ρ creates the Leverage Effect (volatility spikes when markets drop), increasing the probability of breaching the 60% Knock-In barrier compared to a log-normal model.

3.2 FFT-Based Calibration (Carr-Madan Method)

Calibration minimizes the error between model and market prices. To avoid costly numerical integration, we use the Carr-Madan (1999) method, which exploits the analytical characteristic function $\phi_T(u) = \mathbb{E}^{\mathbb{Q}}[e^{iu \ln S_T}]$:

$$\phi_T(u) = \exp\left(C(u,T) + D(u,T)v_0 + iu \ln(S_0 e^{(r-q)T}) \right) \tag{2}$$

Using a damping factor α to ensure square-integrability, the Call price is recovered via the inverse Fourier transform:

$$C(k,T) = \frac{e^{-\alpha k}}{\pi} \int_0^\infty e^{-ivk} \frac{e^{-rT}\phi_T(v-(\alpha+1)i)}{\alpha^2 + \alpha - v^2 + i(2\alpha+1)v} dv \tag{3}$$

This integral is discretized and solved via the Fast Fourier Transform (FFT) ($N = 4096$), enabling rapid pricing. We employ a hybrid optimization strategy combining *Differential Evolution* (Global Search) and *L-BFGS-B* (Local Refinement) to robustly identify the global minimum Θ^*.

3.3 Monte Carlo Pricing: Full Truncation & Path Recycling

The path-dependent nature of the Autocall requires Monte Carlo simulation. A key challenge is the Feller condition violation ($2\kappa\theta < \sigma^2$), which allows variance to become negative in discrete steps. We address this via the Full Truncation scheme (Lord et al., 2006), where the auxiliary variance $\tilde{v}_t$ absorbs negative drifts but is truncated to zero in the diffusion term:

$$\begin{aligned} \tilde{v}_{t+\Delta t} &= \tilde{v}_t + \kappa\left(\theta - \tilde{v}_t^+\right)\Delta t + \sigma\sqrt{\tilde{v}_t^+}\sqrt{\Delta t} Z_v \\ v_{t+\Delta t} &= \max(0, \tilde{v}_{t+\Delta t}) \end{aligned} \tag{4}$$

Finally, to ensure smooth Greek calculations (Delta, Gamma), we use Path Recycling (Common Random Numbers), applying identical Brownian increments to shifted paths to isolate structural sensitivity from stochastic noise.

4 Calibration Results

4.1 Optimization Strategy

Calibrating the Heston model is a non-convex problem prone to local minima. To ensure robust convergence, we employ a two-stage hybrid strategy. First, we use Differential Evolution (Storn & Price, 1997) for a global stochastic search to identify the basin of attraction. Second, we apply the L-BFGS-B algorithm for local refinement to pinpoint the optimal parameters Θ^*. This minimizes the aggregate RMSE between FFT model prices and market quotes while strictly respecting boundary constraints.

4.2 Optimized Parameters and Stability

The calibration converges to the parameters in Table 1. The correlation $\rho \approx -0.72$ confirms the strong leverage effect required to price the downside Knock-In risk.

Param	κ (Speed)	θ (Long Var)	σ (Vol-Vol)	ρ (Corr)	v_0 (Init)
Value	**2.4095**	**0.0548**	**1.1947**	**-0.7226**	**0.0310**

Table 1: Calibrated Heston Parameters (Dec 3, 2025)

Feller Condition Violation: We observe $2\kappa\theta = 0.2641 < \sigma^2 = 1.4273$. This violation indicates the variance process can hit zero, validating our choice of Full Truncation in the Monte Carlo engine to ensure numerical stability.

4.3 Goodness of Fit and Validation

The model's performance is validated against market data (see Appendix A):

- **Long-Term Fit ($T > 3y$):** As shown in Figure 5, the model achieves an excellent fit for the maturities relevant to the Autocallable Note, ensuring an accurate baseline valuation.
- **Short-Term Limits:** For short maturities ($T < 0.5y$), the model underestimates the extreme skew steepness (Figure 4 and Heatmap 6). This is a known limitation of diffusion models lacking jumps, but the impact on this long-dated product is negligible.

Finally, a sanity check on European sensitivities (Figure 7) confirms the calibrated parameters produce coherent Delta and Vega profiles compared to Black-Scholes benchmarks, ensuring the consistency of the pricing engine.

5 Pricing and Risk Analysis

5.1 Fair Value and Lifecycle Analysis

Using the calibrated Heston parameters and the bootstrapped yield curve, we perform a Monte Carlo simulation ($N = 50,000$, daily steps) to value the Note. This yields a theoretical fair value of **$988.53** (98.85% of nominal).

As detailed in Table 2, the valuation represents a trade-off. The discount to par (1.15%) indicates that the funding cost of the embedded downside protection (Short Put) currently outweighs the present value of the expected coupons.

Component	Value ($)	Financial Mechanism
Principal	947.81	Zero-Coupon Bond + Survival Probability ($S_T > 60\%$)
Coupon	+73.79	Accumulation of potential coupons (Yield)
Short Put	-33.07	Funding leg: Capital-at-risk below 60% barrier
Total	**988.53**	**Fair Value**

Table 2: Fair Value Decomposition

Additionally, the simulation estimates an **Expected Life of 1.63 years**. This is significantly shorter than the 4-year maturity, reflecting a high probability of early Autocall given the current spot level relative to the strike ($S_0 = S_{ref}$).

5.2 Component Decomposition

The Note functions as an aggregate of three distinct instruments, each with unique sensitivities (see Figure 1):

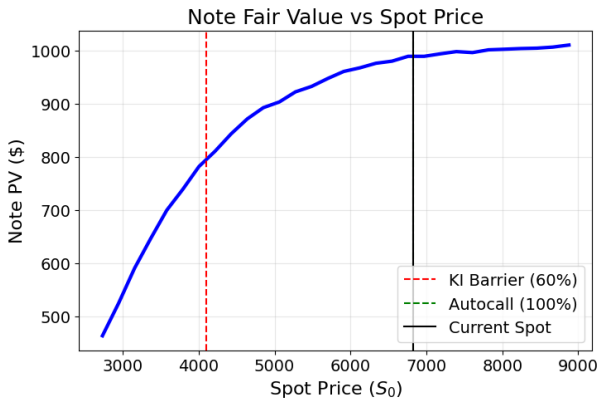


Figure 1: Note Fair Value vs. Spot Price. The vertical Red dashed line represents the 60% Knock-In Barrier, and the Black solid line represents the current Spot Price. The profile shows a limited upside (capped by Autocall) and a sharp downside risk below the barrier.

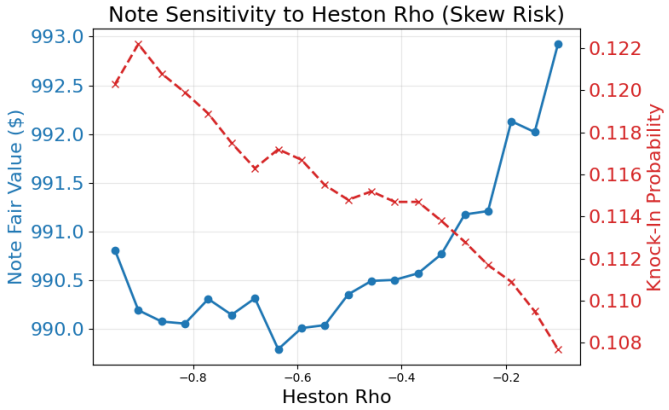


Figure 2: Note Sensitivity to Heston Rho. **Left Axis (Blue):** Fair Value decreases as correlation becomes more negative (Short Skew). **Right Axis (Red):** The probability of triggering the Knock-In barrier increases with negative correlation.

1. Principal Component (Defensive): Above the 60% barrier, this component behaves like a risk-free zero-coupon bond, converging to \$1,000. Below 60%, it collapses linearly. Our logs show that at distressed spot levels ($S_0 \approx 3000$), the Short Put component generates massive losses (−\$428), effectively eating into the principal protection.

2. Coupon Component (Convex): This exhibits a bell-shaped profile. The value peaks where the spot is high enough to accrue coupons but low enough to avoid an immediate call. Once the spot exceeds the Autocall barrier (100%), the component value drops as the product is called at $t = 1$, eliminating future coupon accumulation.

5.3 Greeks and Sensitivity Profile

The risk analysis reveals the structural complexity of the note, characterized by non-monotonic Greeks and radical convexity shifts (see Appendix Figure 8).

The Autocall Paradox (Negative Coupon Delta)

Contrary to standard equities, the Note's sensitivity is non-linear. In the Autocall Zone ($S_t > 100\%$), the coupon component's Delta turns **negative**. *Mechanism:* A further rise in spot price increases the certainty of an immediate Autocall. While this secures the first coupon, it destroys the option value of receiving the potentially larger stream of coupons in years 2, 3, and 4.

The Gamma Flip Phenomenon

The Note exhibits a structural change in convexity around the 60% Knock-In barrier. This profile is distinct from standard options and becomes significantly more pronounced near maturity compared to inception (Appendix Figure 9), creating extreme **Pin Risk** and making perfect hedging impossible without slippage.

- **Short Gamma ($S_t > 60\%$):** Above the barrier, the Total Gamma is generally **negative**. As the spot price falls towards the barrier, the delta increases, forcing the hedger to sell into a falling market (destabilizing).
- **Long Gamma ($S_t < 60\%$):** Once the barrier is breached, the structure behaves like a recovery play (Delta drops as spot drops), resulting in a **positive** Gamma profile (stabilizing).

5.4 Skew Risk (Heston Rho Parameter Sensitivity)

A specific feature of Heston is its ability to price Skew Risk via the correlation ρ. Our stress test (Figure 2) confirms the product is **Short Skew**. When correlation becomes more negative (e.g., $\rho \to -0.9$), the implied volatility skew steepens. This fattens the downside tail of the return distribution, increasing the probability of the Knock-In event and reducing the Note's fair value.

5.5 Model Limitations

Two key limitations remain. First, Heston assumes continuous paths, likely underpricing **Gap Risk** (instantaneous market crashes). Second, the model assumes infinite liquidity, ignoring the execution costs required to manage the massive Gamma positions at the barriers.

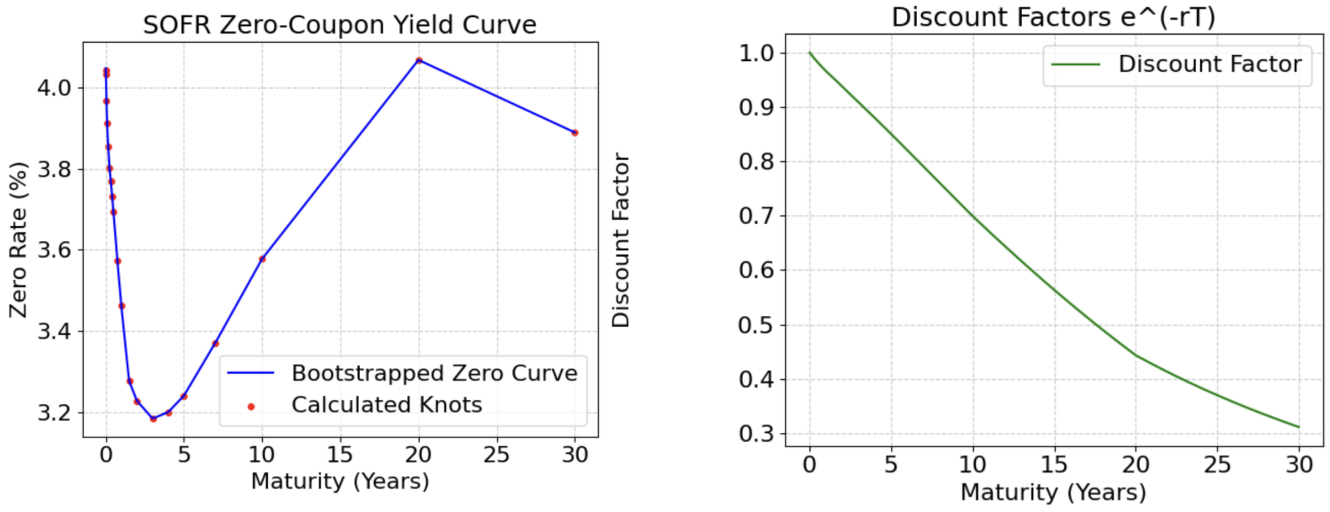
6 Conclusion

This project successfully implemented a pricing engine for the Athena Autocallable Note on the S&P 500. By combining **FFT-based calibration** (Carr-Madan) with a **Full Truncation Monte Carlo** scheme, we achieved a valuation framework that is consistent with market observables.

The calibration process yielded a high-fidelity fit for the product's 4-year maturity ($\rho \approx -0.72$), resulting in a theoretical fair value of **98.85%**. The subsequent risk analysis highlighted the structural complexity of the note, specifically the **Gamma Flip** phenomenon at the barriers and the product's inherent **Short Skew** position.

However, model risk remains the ultimate constraint. While the Heston model captures the stochastic nature of volatility, its diffusive nature fails to account for instantaneous market crashes. Therefore, the calculated fair value should be interpreted as a theoretical baseline, which in a live trading environment would require an additional **safety margin** to cover liquidity costs and unhedgeable jump risks.

A Figures



Zero-Coupon Rates (%) Discount Factors e^{-rT}

Figure 3: **Bootstrapping Results (Nov 25, 2025.)** The left panel displays the constructed SOFR Zero curve; the red dots represent the solved knots from market instruments, while the blue line represents the interpolation used by the pricing engine. The right panel shows the corresponding discount factors used to value future payoffs.

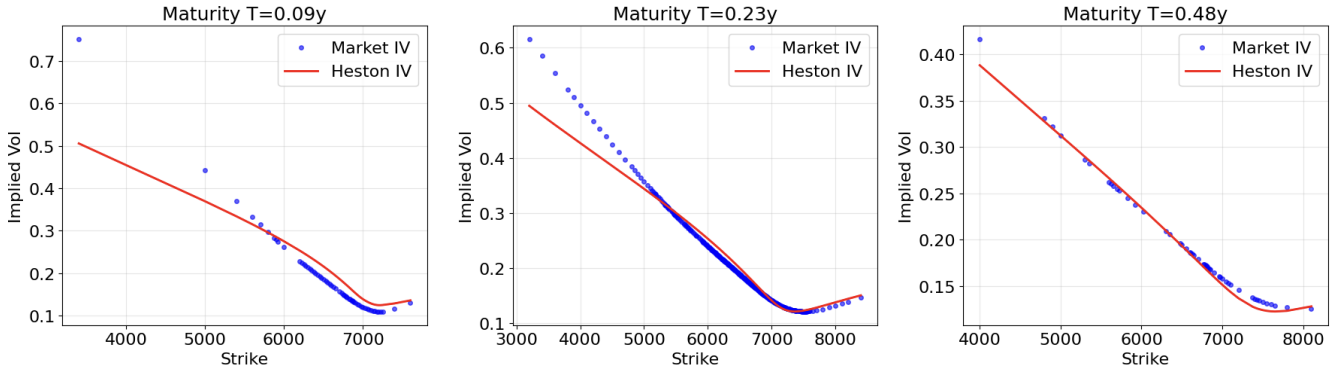


Figure 4: Short-Term Calibration: Market (Blue) vs. Heston (Red). The model captures the general trend but underestimates the extreme skew steepness for $T < 3$ months.

A.1 Price Calibration Overview

Figure 10 details the direct comparison between Market Prices (Blue dots) and Heston Model Prices (Red line) across all calibrated maturities.

Note on Percentage Errors: The annotations showing high percentage errors (e.g., 107.7% at $T = 0.088y$) strictly correspond to deep Out-of-the-Money options where the market price is near zero (e.g., < \$0.05). In these cases, a tiny absolute difference results in a massive relative error, which is numerically expected and financially negligible for the structured product valuation.

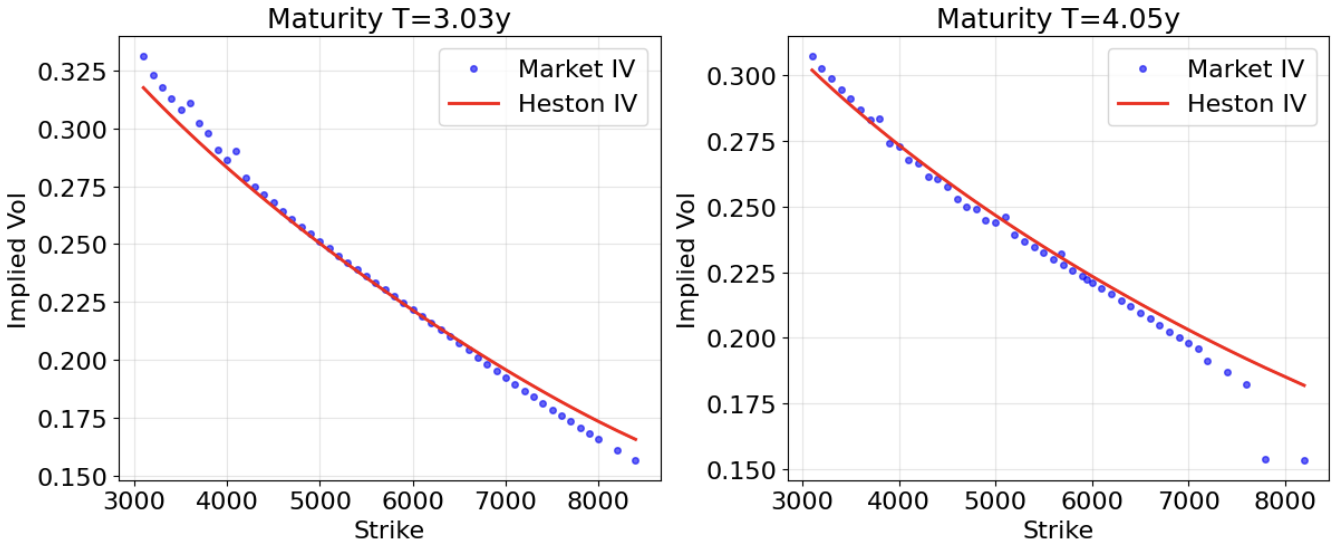


Figure 5: Long-Term Calibration ($T \approx 4$ Years). The model's fit for the relevant product maturity validates the pricing engine, although the fit is slightly sacrificed by medium-maturity data points.

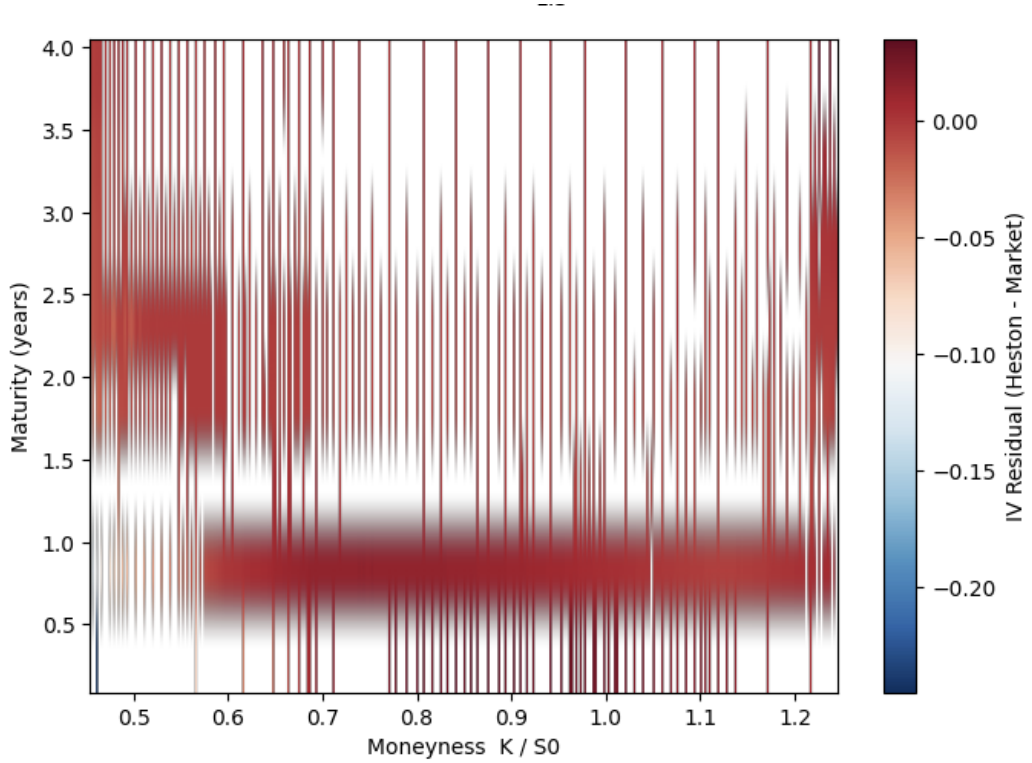


Figure 6: Implied Volatility Residual Heatmap. The **Blue** area (bottom-left) identifies the underestimation of short-term skew, while the **Red** area covers the stable long-term region.

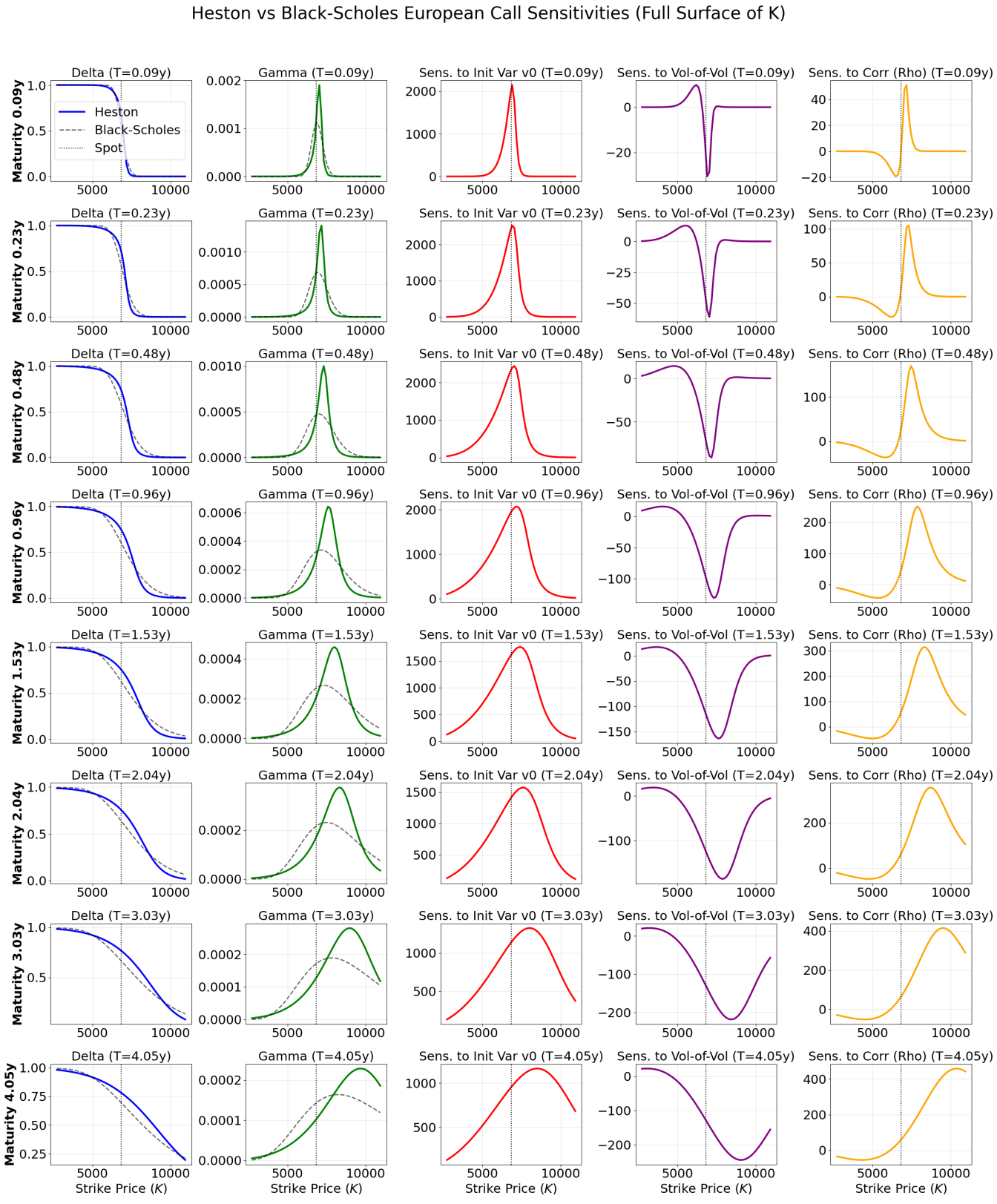


Figure 7: Heston vs. Black-Scholes Greeks for European Calls. The coherence of these profiles validates the stability of the calibrated parameters before applying them to the exotic structure.

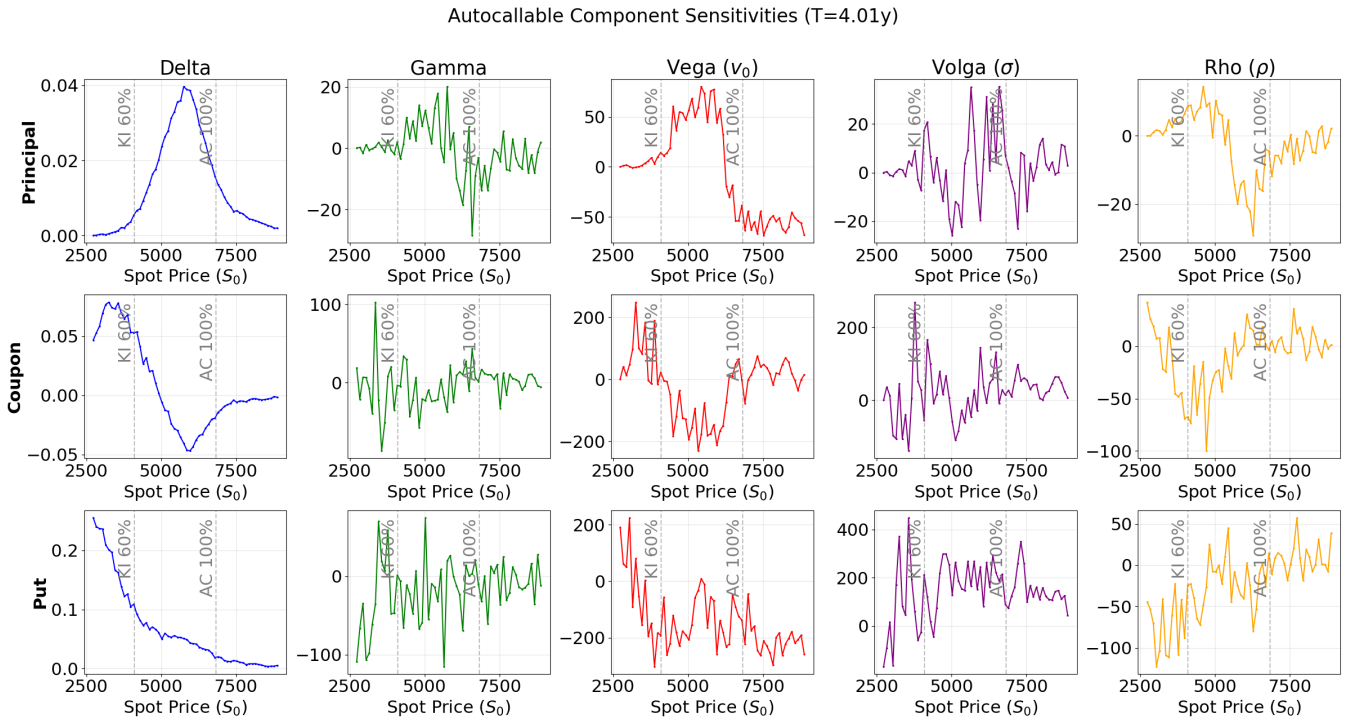


Figure 8: Component Sensitivities at Inception ($T = 4y$). The matrix displays the impact of each Heston parameter on the Principal, Coupon, and Put components. Note the Coupon Delta (row 2, col 1) turning negative at high spots, illustrating the Gamma Flip and Autocall Paradox.

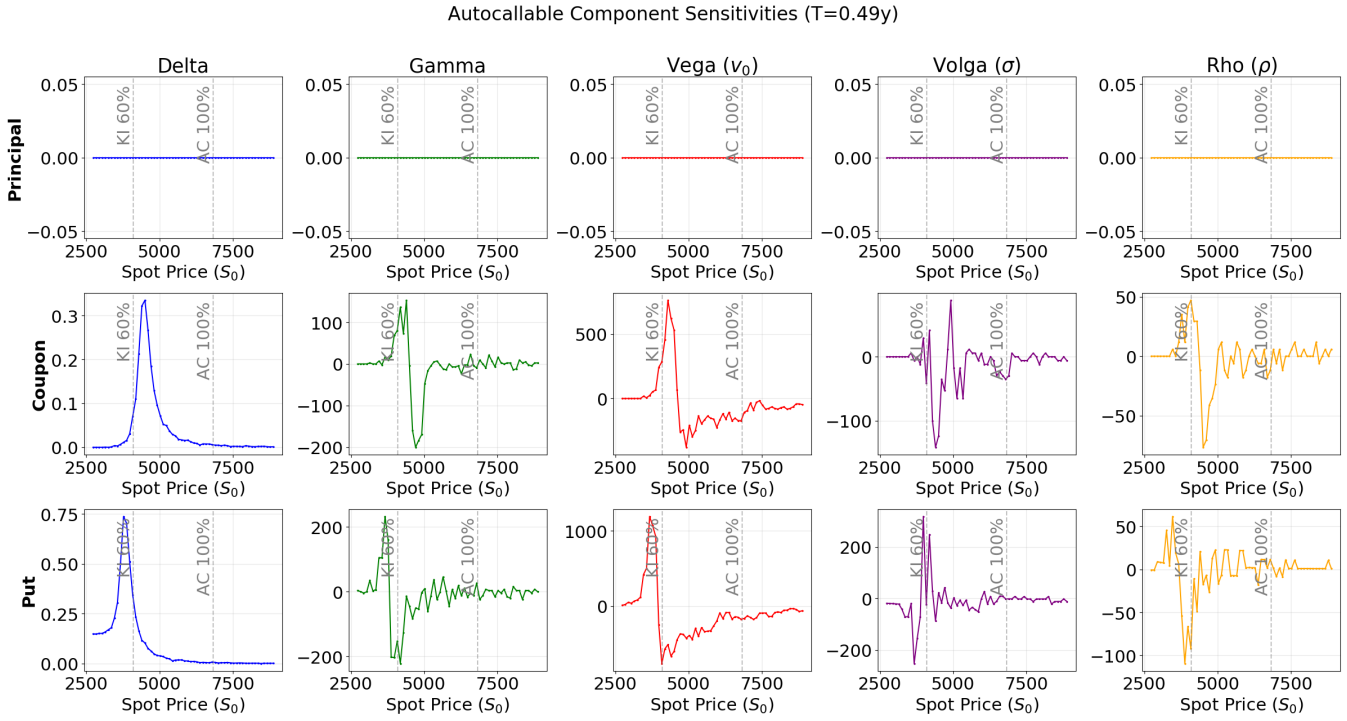


Figure 9: Component Sensitivities near Maturity ($T = 0.49y$). Unlike the long-term profile, the risks here are no longer diffuse but concentrated strictly at the barriers (Knock-In at 60% and Autocall at 100%). This visualizes the Digital Risk and the difficulty of hedging near observations.

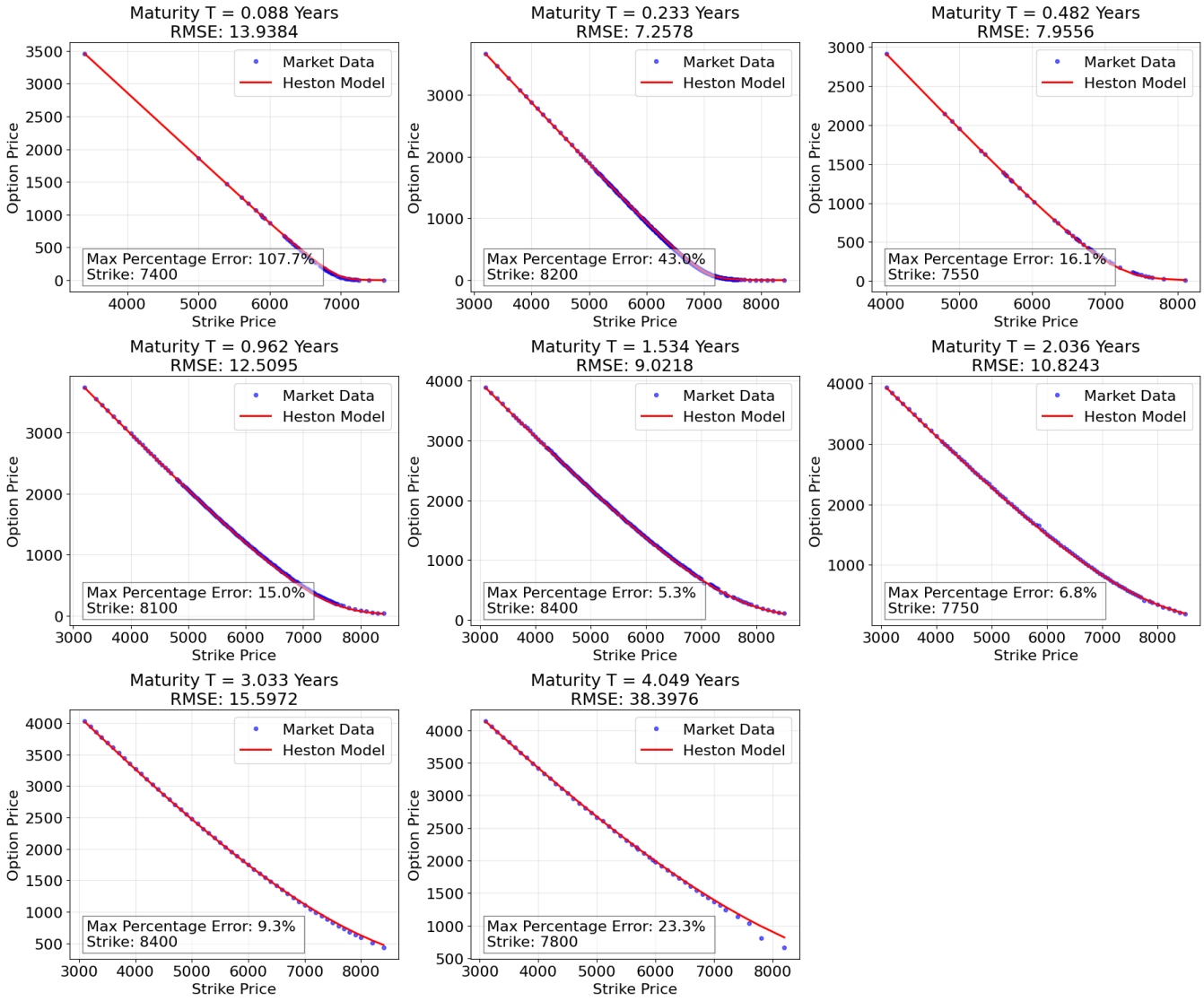


Figure 10: Model vs. Market Prices across Maturities. The Heston model demonstrates a robust fit across most of the term structure, particularly for the longer maturities ($T > 3y$) relevant to the Autocallable Note.

B Main Python Implementation

Below is part of our python implementation. We use the Github repo by cantaro86 as our main tutorial and reference for calibration and path simulation.

```python
import numpy as np
import pandas as pd
from datetime import datetime, timedelta
import copy
import os
import sys
from contextlib import contextmanager
from scipy.fft import ifft
from scipy.interpolate import interp1d
```

```python
from scipy.optimize import minimize, differential_evolution, OptimizeResult
from numba import njit
```

Listing 1: Libraries

```python
class OISBootstrapper:
    def __init__(self):
        """Initializes with hardcoded Bloomberg SOFR OIS par swap rates on Nov 25, 2025, sorted by maturity."""
        self.market_data = [
            (1/52, 4.0433), (2/52, 4.0353), (3/52, 3.9704),
            (1/12, 3.9175), (2/12, 3.8650), (3/12, 3.8182),
            (4/12, 3.7930), (5/12, 3.7605), (6/12, 3.7283),
            (9/12, 3.6226), (1,  3.5245),
            (1.5,  3.3602), (2,  3.2860), (3,  3.2410),
            (4,  3.2557), (5,  3.2940), (7,  3.4145),
            (10, 3.6005), (20, 3.9565), (30, 3.9435),
        ]
        self.market_data.sort(key=lambda x: x[0])

    def bootstrap(self):
        """Calculates the zero-coupon curve from market data.
        Methodology:
        - T < 2 years: Uses simple interest discounting.
        - T >= 2 years: Uses iterative bootstrapping assuming annual fixed coupons.
        - Interpolation: Linear interpolation of zero rates."""

        known_ts = [] # maturities successfully calculated
        known_rs = [] # annualized, continuously compounded zero-coupon rates for known_ts

        # Helper for linear interpolation of zero rates
        def get_df(t):
            if t <= 0:
                return 1
            # If t is beyond our current knowledge, use the last known zero rate (flat extrapolation)
            if not known_ts:
                return 1

            # Interpolate R_zero, NOT DF
            r = np.interp(t, known_ts, known_rs)
            return np.exp(-r * t)

        for T, rate_pct in self.market_data:
            R_par = rate_pct / 100
```

```
            if T < 2: # 18M max
                # Short tenor: simple discounting
                df_T = 1 / (1 + R_par * T)
                r_zero = -np.log(df_T) / T

            else:
                # Long tenor
                # First calculate value of the fixed coupons before maturity
                prev_coupons_val = 0
                payment_dates = np.arange(1, int(T) + 0.01)
                payment_dates = payment_dates[payment_dates < T - 0.01]
                for t_pay in payment_dates:
                    prev_coupons_val += R_par * get_df(t_pay)
                # Boorstap from discounted principal_fixed coupon value at maturity
                df_T = (1 - prev_coupons_val) / (1 + R_par)
                r_zero = -np.log(df_T) / T

            known_ts.append(T)
            known_rs.append(r_zero)

        return interp1d(known_ts, known_rs, kind='linear', fill_value=extrapolate)

# Instantiate global curve for module-level access
_bootstrapper = OISBootstrapper()
sofr_zero_curve = _bootstrapper.bootstrap()

# Define rate curve function (wrapper)
def rate_curve_func(T):
    """Returns annualized zero rate for T"""
    if isinstance(T, (np.ndarray, list, pd.Series)):
        T = np.maximum(T, 1e-4) # Avoid T=0 division
    else:
        if T <= 0:
            T = 1e-4
    return sofr_zero_curve(T)

# Define Discount Factor Function
def DF(T):
    """Calculates discount factor e^(-rT)"""
    r = rate_curve_func(T)
    return np.exp(-r * T)
```

Listing 2: Bootstrapping and Curves

```python
class HestonFFTCalibrator:
    def __init__(self, market_data, S0, r_curve_func):
        """Lewis (2001) FFT Calibrator with Hybrid Optimization (Global + Local)."""
        self.market_data = market_data.copy()
        self.S0 = S0
        self.r_curve = r_curve_func  # Store the function

        # Optimization Bounds [kappa, theta, sigma, rho, v0]
        self.bounds = [
            (0.01, 15),   # kappa: Mean reversion speed
            (0.001, 1),   # theta: Long-run variance
            (0.01, 10),    # sigma: Vol of vol
            (-0.999, 0.999),# rho: Correlation (avoid exact -1/1)
            (0.001, 1)    # v0: Initial variance
        ]

        # Grid Parameters
        self.N = 4096
        self.B = 300

        self.du = self.B / self.N
        self.u = np.arange(self.N) * self.du

        self.dk = 2 * np.pi / self.B
        self.b = self.N * self.dk / 2
        self.k_fft = -self.b + self.dk * np.arange(self.N)

        # Simpson's Weights
        self.weights = 3 + (-1)  (np.arange(self.N) + 1)
        self.weights[0] = 1
        self.weights[self.N - 1] = 1
        self.weights = self.weights * self.du / 3

        # Pre-Group Data
        self.unique_T = self.market_data['T'].unique()
        self.data_batches = []

        for T in self.unique_T:
            mask = self.market_data['T'] == T
            batch = {
                'T': T,
                'strikes': self.market_data.loc[mask, 'strike'].values,
                'market_prices': self.market_data.loc[mask, 'market_price_call_equivalent'].values,
```

```
                'k_market': np.log(S0 / self.market_data.loc[mask, 'strike'].values),
                'q': self.market_data.loc[mask, 'implied_q'].iloc[0]
            }
            self.data_batches.append(batch)

    def _heston_char_func(self, u, T, r_t, q_t, params):
        """Stable Albrecher/Schoutens form Heston characteristic function."""
        kappa, theta, sigma, rho, v0 = params

        # Drift uses the specific rate for this T
        mu = r_t - q_t

        xi = kappa - sigma * rho * u * 1j
        d = np.sqrt(xi2 + sigma2 * (u2 + 1j * u))
        g1 = (xi + d) / (xi - d)
        g2 = 1 / g1

        cf = np.exp(
            1j * u * mu * T
            + (kappa * theta) / (sigma2) * ((xi - d) * T - 2 * np.log((1 - g2 * np.exp(-d * T)) / (1 - g2
))))
            + (v0 / sigma2) * (xi - d) * (1 - np.exp(-d * T)) / (1 - g2 * np.exp(-d * T))
        )
        return cf

    def price_surface(self, params):
        """FFT Discretized Formula. Calculates Heston model prices for each strike and maturity."""
        model_prices = []

        for batch in self.data_batches:
            T = batch['T']
            r_t = self.r_curve(T)
            q_t = batch['q']

            # Characteristic Function
            cf_vals = self._heston_char_func(self.u - 0.5j, T, r_t, q_t, params)

            # Input Array
            input_array = np.exp(-1j * self.b * self.u) * cf_vals * self.weights / (self.u2 + 0.25)

            # Inverse FFT
            integral_val = np.real(ifft(input_array) * self.N)

```

```
            # Interpolation
            spline = interp1d(self.k_fft, integral_val, kind='cubic', fill_value=extrapolate)
            integral_interp = spline(batch['k_market'])

            # Lewis Formula
            discount = np.exp(-r_t * T)
            div_factor = np.exp(-q_t * T)

            prices = (self.S0 * div_factor) - (np.sqrt(self.S0 * batch['strikes']) * discount / np.pi) *
integral_interp
            model_prices.extend(prices)

        return np.array(model_prices)

    def objective_function(self, params):
        """RMSE objective."""
        try:
            model_vals = self.price_surface(params)
        except:
            return 1e10 # Catch numerical overflows

        market_vals = np.concatenate([b['market_prices'] for b in self.data_batches])
        mse = np.mean((model_vals - market_vals)2)
        return np.sqrt(mse)

    def calibrate(self, verbose=True, fixed_params=None):
        """Global optimization with local refinement.

        1. Global Search: Uses Differential Evolution to find a candidate solution.
        2. Local Refinement: Uses L-BFGS-B starting from the global best to minimize RMSE.

        Fixed-parameter option for testing: skips optimization and returns the RMSE directly."""
        if fixed_params is not None:
            if verbose:
                print(f"Skipping optimization. Using fixed parameters: {fixed_params}")
            # Create a fake result object that looks exactly like scipy's output
            fixed_array = np.array(fixed_params)
            # Optional: Calculate the RMSE for these parameters so we can see the error metric
            rmse = self.objective_function(fixed_array)
            return OptimizeResult(x=fixed_array, fun=rmse, message=Fixed Parameters Used)

        if verbose: print("Starting global optimization (Differential Evolution).")

```

```
        # Global Optimization
        result_global = differential_evolution(
            self.objective_function,
            bounds=self.bounds,
            strategy='best1bin',
            maxiter=50,      # Cap iterations to save time
            popsize=15,      # Population size
            tol=1e-4,
            workers=-1,       # Parallelize
            seed=42
        )

        if verbose:
            print(f"Global Best: {result_global.x}, RMSE: {result_global.fun:.4f}")
            print("Starting Local Optimization (L-BFGS-B).")

        # Local Refinement: use the global result as the starting point (x0)
        result_local = minimize(
            self.objective_function,
            x0=result_global.x,
            method='L-BFGS-B',
            bounds=self.bounds,
            tol=1e-6
        )

        if verbose:
            print(f"Local Best: {result_local.x}, RMSE: {result_local.fun:.4f}")

        return result_local
```

Listing 3: Heston Calibrator

```
# Define static numba function outside of the class to speed up simulation
@njit(fastmath=True)
def heston_engine_numba(S0, v0, kappa, theta, sigma, rho,
                        dt, sqrt_dt, Z1, Z2, r_drift_scalar):
    """Standalone Heston simulation engine compiled by Numba for speed."""
    N_steps, n_paths = Z1.shape

    # Pre-allocate
    S_paths = np.zeros((N_steps + 1, n_paths))
    v = np.zeros((N_steps + 1, n_paths))

    S_paths[0, :] = S0
```

```
    v[0, :] = v0

    # Simulation Loop (C-Speed)
    for t in range(N_steps):
        v_curr = v[t, :]
        v_pos = np.maximum(0.0, v_curr)

        # Variance Step (Full Truncation)
        v[t+1, :] = v_curr + kappa * (theta - v_pos) * dt + \
                    sigma * np.sqrt(v_pos) * sqrt_dt * Z1[t, :]

        # Stock Step
        # Pre-calculating the correlated noise term
        W_S = rho * Z1[t, :] + np.sqrt(1 - rho2) * Z2[t, :]

        S_paths[t+1, :] = S_paths[t, :] * np.exp(
            (r_drift_scalar - 0.5 * v_pos) * dt +
            np.sqrt(v_pos) * sqrt_dt * W_S
        )

    return S_paths, v # Return both if need volatility paths later

#---------------------Heston Pricing Engine (Including Sensitivities)---------------------
class HestonAutocallPricer:
    def __init__(self, S0, S_ref, r_curve_func, q_curve_func, params):
        """Heston Autocall Pricer."""
        self.S0 = S0
        self.S_ref = S_ref
        self.r_curve_func = r_curve_func
        self.q_curve_func = q_curve_func
        self.kappa, self.theta, self.sigma, self.rho, self.v0 = params

        # State variables to hold simulation data
        self.time_grid = None
        self.S_paths = None
        self.is_simulated = False
        self.expected_life = 0

        # Sensitivities: delta, gamma, and sensitivities to v0/sigma/rho
        self.sensitivities = {}


    def run_simulation(self, T_max, steps_per_year=252, n_paths=50000, force_unit_start=False):
```

```
        """Runs the Heston Euler-Maruyama simulation with Full Truncation.
        Stores the time_grid and paths internally."""
        # Get flat rate for the simulation horizon drift
        r_val = self.r_curve_func(T_max)
        q_val = self.q_curve_func(T_max)
        r_drift = r_val - q_val

        # Setup grid: calculate the daily steps needed to fit into the remaining time
        N_steps = int(np.ceil(T_max * steps_per_year))
        self.time_grid = np.linspace(0, T_max, N_steps + 1)
        dt = T_max / N_steps
        sqrt_dt = np.sqrt(dt)

        # Correlated Brownian motions
        Z1 = np.random.normal(size=(N_steps, n_paths)) # Variance noise
        Z2 = np.random.normal(size=(N_steps, n_paths)) # Stock noise (uncorrelated)
        W_v = Z1
        W_S = self.rho * Z1 + np.sqrt(1 - self.rho2) * Z2

        # Initialize arrays
        self.S_paths = np.zeros((N_steps + 1, n_paths))
        v = np.zeros((N_steps + 1, n_paths))

        # If forcing unit start, we simulate starting at 1.0 regardless of self.S0
        if force_unit_start:
            start_val = 1.0
        else:
            start_val = self.S0

        self.S_paths[0] = start_val
        v[0] = self.v0

        # Time stepping (Euler-Maruyama)
        # Unwrap 'self' variables and pass them as simple arguments
        self.S_paths, _ = heston_engine_numba(
            start_val, self.v0,
            self.kappa, self.theta, self.sigma, self.rho,
            dt, sqrt_dt,
            Z1, Z2,
            r_drift
        )

        self.is_simulated = True
```

```
        print(f"Simulation complete: {n_paths} paths over {T_max:.2f} years ({N_steps} steps).")

    def _get_time_index(self, T_target):
        """Finds the closest simulation index for a requested time T."""
        if not self.is_simulated:
            raise RuntimeError(Run simulation first!)
        idx = np.searchsorted(self.time_grid, T_target)
        if idx >= len(self.time_grid):
            idx = len(self.time_grid) - 1
            print(f"Observation date {T_target:.2f}y is beyond simulation. Using index {idx} (Time {self.time_grid[-1]:.2f}y).")
        return idx

    def price_structure(self, full_schedule,
                        coupon_step=0.0452,
                        barrier_redemption=0.70,
                        barrier_knock_in=0.60,
                        exit_bonus=0.3616,
                        nominal=1000,
                        pricing_date=None):

        """Live Pricer with flexible product parameters. All rates in decimal!
        Decomposes the Fair Value into:
        1. Principal (Bond)
        2. Coupons (Digital Calls)
        3. Risk (Short Put)"""


        if not self.is_simulated:
            raise RuntimeError(Please run .run_simulation() before pricing.)
        # Note: Frist autocall observation date is one year from TODAY

        # Filter future dates
        if pricing_date is not None:
            today = pricing_date
        else:
            today = datetime.now()
        future_times = []
        future_period_indices = []

        for i, date_str in enumerate(full_schedule):
            d = datetime.strptime(date_str, %Y-%m-%d)
            if d > today:
```

```
                T_rem = (d - today).days / 365
                future_times.append(T_rem)
                # i=0 is pricing_date. If it's TODAY,
                # First Exit Rate is 2 * coupon_step in accordance with Page 19.
                future_period_indices.append(i + 2)

        if not future_times:
            return 0

        # Setup barriers (based on S_ref)
        B_autocall = 1 * self.S_ref
        B_redemption = barrier_redemption * self.S_ref
        B_knock_in = barrier_knock_in * self.S_ref

        n_paths = self.S_paths.shape[1]
        active_mask = np.ones(n_paths, dtype=bool)

        # Setup termination tracking: initialize all paths to terminate at maturity
        # If they autocall early, update this array
        final_maturity_T = future_times[-1]
        termination_times = np.full(n_paths, final_maturity_T)

        # Setup path pv's for three components; add up to note pv
        pv_principal = np.zeros(n_paths)
        pv_coupons   = np.zeros(n_paths)
        pv_risk      = np.zeros(n_paths)

        # Loop through all future observation dates except maturity
        for t_obs, n_period in zip(future_times[:-1], future_period_indices[:-1]):

            idx = self._get_time_index(t_obs)
            S_current = self.S_paths[idx, :]

            # Check autocall
            did_autocall = active_mask & (S_current >= B_autocall)

            # Record the current time if autocalled
            termination_times[did_autocall] = t_obs

            # Discount
            r_t = self.r_curve_func(t_obs)
            df = np.exp(-r_t * t_obs)

```

```
            # Payoff split: principal, nominal; coupons, nominal * coupon; add up to nominal * (1 + coupon)
            pv_principal[did_autocall] = nominal * df
            pv_coupons[did_autocall]   = (nominal * n_period * coupon_step) * df

            active_mask[did_autocall] = False

        # Observation at maturity
        idx_final = self._get_time_index(future_times[-1])
        S_final = self.S_paths[idx_final, :]
        T_final = future_times[-1]

        r_mat = self.r_curve_func(T_final)
        df_final = np.exp(-r_mat * T_final)

        survivors = active_mask

        # Scenario A: redemption condition met
        # Payoff is 100% nominal (principal) + Exit coupon
        mask_A = survivors & (S_final >= B_redemption)
        pv_principal[mask_A] = nominal * df_final
        pv_coupons[mask_A]   = (nominal * exit_bonus) * df_final

        # Scenario B: safe zone
        # Payoff is 100% nominal only
        mask_B = survivors & (S_final >= B_knock_in) & (S_final < B_redemption)
        pv_principal[mask_B] = nominal * df_final

        # Scenario C: knock-in loss
        # Payoff is the underlying price, which can be decomposed into:
        # 100% nominal + nominal * (S_final/S_ref - 100%); since S < S_ref, this term is negative
        mask_C = survivors & (S_final < B_knock_in)
        performance = S_final / self.S_ref
        pv_principal[mask_C] = nominal * df_final
        pv_risk[mask_C]      = nominal * (performance[mask_C] - 1) * df_final

        # Calculate expected life
        self.expected_life = np.mean(termination_times)

        # Aggregate
        avg_principal = np.mean(pv_principal)
        avg_coupons   = np.mean(pv_coupons)
        avg_risk      = np.mean(pv_risk)
```

```
226
227         total_price = avg_principal + avg_coupons + avg_risk
228         print(f"Autocallable note price: {total_price:.2f}, \
229         \nPrincipal component: {avg_principal:.2f}, \
230         \nCoupon component: {avg_coupons:.2f}, \
231         \nPut risk component (short down-and-in put): {avg_risk:.2f}")
232
233         return (total_price, avg_principal, avg_coupons, avg_risk)
```

Listing 4: Heston Pricing Engine (Numba Optimized)

References

[1] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.

[2] BNP Paribas Issuance B.V. Final Terms dated 27 April 2023: Issue of USD 35,000,000 Athena Autocall Standard Notes linked to the S&P 500 Index due 27 April 2027, April 2023. ISIN: XS2490595951. Guarantor: BNP Paribas.

[3] Daniele Cantaro. Financial-models-numerical-methods. https://github.com/cantaro86/Financial-Models-Numerical-Methods, 2020. Accessed: 2025-12-19.

[4] Peter Carr and Dilip Madan. Option valuation using the fast fourier transform. *Journal of Computational Finance*, 2(4):61–73, 1999.

[5] Walter Farkas, Francesco Ferrari, and Urban Ulrych. Pricing autocallables under local-stochastic volatility. *Frontiers of Mathematical Finance*, 1(4):575–610, 2022.

[6] Steven L. Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *The Review of Financial Studies*, 6(2):327–343, 1993.

[7] Alan L. Lewis. A simple option formula for general jump-diffusion and other exponential lévy processes. *Available at SSRN 282113*, 2001.

[8] Roger Lord, Remmert Koekkoek, and Dick van Dijk. A comparison of biased simulation schemes for stochastic volatility models. Technical Report 2006-046/4, Tinbergen Institute Discussion Paper, 2006.

[9] Wim Schoutens, Erwin Simons, and Jurgen Tistaert. A perfect calibration! now what? *Wilmott Magazine*, pages 66–78, March 2003.

[10] Rainer Storn and Kenneth Price. Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4):341–359, 1997.